

CC3092 - Deep Learning y Sistemas Inteligentes

Laboratorio #6

Transfer Learning y Fine-Tuning

Instrucciones generales

- En parejas o individual.
- Entrega: miércoles 30 de septiembre, 2026. 23:59.

1. Dataset

Trabjarán con el dataset público CIFAR-10, un problema de clasificación multiclase de imágenes a color en 10 categorías. Cada imagen es de 32×32 píxeles.

Dataset con torchvision:

```
from torchvision import datasets
train = datasets.CIFAR10(root="data", train=True, download=True)
test = datasets.CIFAR10(root="data", train=False, download=True)
```

2. Exploración y preparación de los datos

Cargue el dataset y responda las siguientes preguntas:

- ¿Cuántas observaciones y cuántas clases tiene el dataset? ¿Las clases están balanceadas?
- ¿Cuál es la resolución y el número de canales de las imágenes? Calcule la media y desviación estándar por canal del conjunto de entrenamiento y compárelas con las estadísticas de normalización de ImageNet.
- Visualice al menos 5 ejemplos por clase. ¿Qué pares de clases esperan que sean más difíciles de distinguir y por qué?
- ¿Qué transformaciones de preprocesamiento necesita cada modelo? Defina un pipeline para la CNN desde cero (resolución nativa) y otro para VGG-16 (redimensionamiento y normalización de ImageNet). ¿Qué pasa con el número de píxeles, y por lo tanto con el cómputo, al pasar de 32×32 a 224×224 ?
- ¿Qué técnicas de data augmentation utilizarán y por qué? Justifique por qué la augmentation se aplica solo al conjunto de entrenamiento.

Divida el conjunto de entrenamiento original en entrenamiento y validación (por ejemplo, 45,000 / 5,000, estratificado por clase) y reserve el conjunto de test oficial para la evaluación final.

3. Investigación: transfer learning en PyTorch

Investigue las clases, utilidades y herramientas necesarias para reutilizar un modelo preentrenado y para medir su costo computacional. Para cada una, describa brevemente su propósito y sus parámetros más relevantes.

- torchvision.models.vgg16 y VGG16_Weights (incluyendo weights.transforms())
- Estructura del modelo: model.features, model.avgpool (nn.AdaptiveAvgPool2d) y model.classifier
- requires_grad y cómo congelar / descongelar capas o bloques completos

- `model.train()` vs. `model.eval()` y `torch.no_grad()`
- Grupos de parámetros en el optimizador (`param_groups`) para usar learning rates distintos por capa
- Una herramienta para contar parámetros y operaciones: `torchinfo`, `fvcore`, `thop` o `ptflops`
- `torch.cuda.max_memory_allocated()` y `torch.cuda.synchronize()` para medir memoria y tiempo en GPU

Adicionalmente, investigue brevemente los siguientes conceptos: diferencia entre MACs y FLOPs y cómo se relacionan; diferencia entre parámetros totales y parámetros entrenables.

4. Construcción y entrenamiento de los modelos

Construya y entrene tres modelos distintos para clasificar las imágenes de CIFAR-10:

- **CNN desde cero:** una red convolucional propia, inicializada aleatoriamente y entrenada sobre las imágenes en su resolución nativa (32×32). Debe tener al menos 3 bloques convolucionales; se recomienda el uso de batch normalization, dropout y data augmentation.
- **VGG-16 como feature extractor:** cargue VGG-16 con pesos de ImageNet, congele todos los bloques convolucionales y reemplace la última capa del clasificador por una capa de 10 salidas. Solo se entrenan los parámetros del clasificador.
- **VGG-16 con fine-tuning:** partiendo del mismo modelo preentrenado, descongele uno o más bloques convolucionales superiores (por ejemplo, el bloque 5, o los bloques 4 y 5) y entrénelos junto con el clasificador, usando un learning rate menor para el backbone que para el clasificador.

Pueden usar una resolución de entrada menor a 224×224 para VGG-16 (por ejemplo, 112×112 o 128×128) si tienen restricciones de cómputo, siempre que sea la misma para ambos modelos basados en VGG-16 y se justifique en el reporte.

Itere sobre cada modelo para obtener la mejor configuración posible. En el caso del fine-tuning, al menos una de las iteraciones debe variar el número de bloques descongelados.

Para cada iteración, registre:

- La configuración de hiperparámetros usada (y, para VGG-16, qué capas están congeladas).
- La pérdida (loss) de entrenamiento y de validación por epoch.
- Las métricas de evaluación del problema de clasificación: accuracy, precision, recall y F1-score (macro), calculadas sobre el conjunto de validación.
- El número de parámetros totales y el número de parámetros entrenables.
- El tiempo de entrenamiento por epoch y el tiempo total, y la memoria pico de GPU.

Grafiquen las curvas de pérdida de entrenamiento y validación de al menos 3 iteraciones por modelo (9 en total), para poder identificar visualmente señales de overfitting o underfitting.

Una vez identificada la mejor configuración de cada modelo según sus métricas de validación, evalúe los tres modelos finales una única vez sobre el conjunto de test, reporte sus métricas y genere la matriz de confusión de cada uno.

4.1 Experimento adicional: efecto de la cantidad de datos

Usando las mejores configuraciones de los tres modelos, entrénelos nuevamente con una fracción reducida del conjunto de entrenamiento (por ejemplo, 10 % de las imágenes, manteniendo el balance entre clases) y evalúenlos sobre el mismo conjunto de test. Comparen el desempeño de cada modelo con el obtenido usando el 100 % de los datos.

5. Comparación de rendimiento y recursos

Con los resultados de la mejor configuración de cada modelo, construyan una tabla comparativa entre la CNN desde cero, VGG-16 como feature extractor y VGG-16 con fine-tuning, que incluya:

- Parámetros totales y parámetros entrenables.
- Costo de inferencia: MACs o FLOPs por imagen (forward pass) y latencia promedio por imagen en GPU.
- Costo de entrenamiento: tiempo por epoch, número de epochs hasta alcanzar la mejor métrica de validación, tiempo total y memoria pico de GPU.
- Estimación del cómputo total de entrenamiento (en FLOPs) a partir del costo por imagen, el número de imágenes y el número de epochs, considerando qué partes de la red realizan backward pass en cada estrategia.
- Desempeño sobre el conjunto de test (accuracy, precision, recall, F1-score).
- Resultados del experimento de cantidad de datos (sección 4.1).

Adicionalmente, generen al menos dos gráficas: (1) accuracy de validación contra tiempo de entrenamiento acumulado, con los tres modelos en la misma figura; y (2) F1-score de test contra costo computacional (FLOPs de inferencia o de entrenamiento). Indiquen en el reporte el hardware utilizado (modelo de GPU / CPU).

6. Discusión y análisis

Responda con base en sus resultados:

- ¿Cuál de los tres modelos obtuvo el mejor desempeño en test? ¿Qué tan grande fue la diferencia y justifica el costo adicional en parámetros, FLOPs y tiempo de entrenamiento?
- ¿Qué modelo ofrece la mejor relación entre desempeño y recursos? Justifiquen su respuesta usando las gráficas de la sección 5.
- VGG-16 fue entrenada con fotografías de 224×224 y CIFAR-10 tiene imágenes de 32×32 . ¿Por qué las features preentrenadas siguen siendo útiles? ¿Qué efecto tiene el redimensionamiento en el desempeño y en el costo?

- En el fine-tuning, ¿cómo afectó el número de bloques descongelados y el learning rate del backbone? ¿Observaron señales de overfitting o de catastrophic forgetting?
- Con base en el experimento de la sección 4.1, ¿cuál modelo se vio más afectado al reducir los datos? ¿Cómo relacionan esto con lo discutido en clase sobre cuándo usar feature extraction, fine-tuning o entrenar desde cero?
- Analicen las matrices de confusión: ¿qué pares de clases se confunden más en cada modelo? ¿Los tres modelos cometen los mismos errores?
- Si tuvieran que desplegar un modelo para este problema en (a) un servidor con GPU y (b) un dispositivo móvil, ¿qué modelo elegirían en cada caso y por qué? ¿Qué arquitectura preentrenada alternativa a VGG-16 considerarían para el caso (b)?

Entregables

Entregable	Contenido
PDF (máx. 4 páginas)	Investigación de transfer learning en PyTorch, tabla de resultados de las iteraciones (CNN desde cero, VGG-16 feature extraction y VGG-16 fine-tuning), tabla comparativa de rendimiento y recursos, análisis de resultados y conclusiones.
Repositorio (Git)	Jupyter Notebook completo y comentado: carga y preparación de datos, investigación, definición de los tres modelos, entrenamiento de las 9+ iteraciones, medición de recursos, experimento de cantidad de datos y evaluación final sobre test.

Incluyan el enlace al repositorio al final del PDF.